CUSTOMER REGISTRY MANAGEMENT SYSTEM

Full-Stack Customer Relationship & Support Desk Console (MERN)

Date: 11 July 2026

1. Introduction

Project Title: CUSTOMER REGISTRY MANAGEMENT SYSTEM (FULL-STACK MERN)

Project Nature: A full-stack MERN web application developed to manage customer records, support tickets, feedback, reminders, and user authentication through a centralized dashboard. The system provides secure customer management, efficient support operations, and scalable RESTful services..

Team Members:

1.NIKHITHASREE– Full Stack Developer (Frontend, Backend, Database, Testing & Documentation).

2. Project Overview

Purpose: To deliver an all-in-one operations console that empowers sales and support teams to register client profiles, log note interactions, resolve ticketing queues, monitor real-time SLA countdown targets, and aggregate feedback satisfaction scores.

Key Features:

- **Secure User Registration & Login:** Allows users to register accounts securely and login with credentials.
- **Customer Management (CRUD):** Enables complete customer lifecycle management, including creating, reading, updating, and deleting customer profiles.
- **Support Ticket Management:** Tracks incoming support requests, assigning agents, and monitoring status progress from open to closed.
- **Customer Interaction History:** Logs chronological history of calls, meetings, and emails for each registered client in a detailed timeline.
- **Feedback Management:** Collects customer satisfaction reviews and rating stars to measure service quality.
- **Reminder Management:** Allows setting follow-up actions and reminders for customer calls or support ticket actions.
- **Dashboard Analytics:** Visualizes customer growth statistics and support agent ticket workloads via interactive charts.
- **Search & Filter Customers:** Supports search queries and status filters on customer registries in real-time.
- **JWT Authentication:** Secures API endpoints and manages user sessions utilizing JSON Web Tokens.

- **MongoDB Database Integration:** Stores customer profiles, support tickets, and agent workloads reliably in MongoDB.
- **Responsive User Interface:** Provides a clean, modern dashboard layout that fits screens of all sizes.
- **RESTful API Architecture:** Coordinates data exchange between Vite/React client and Express backend securely.

Table of Contents

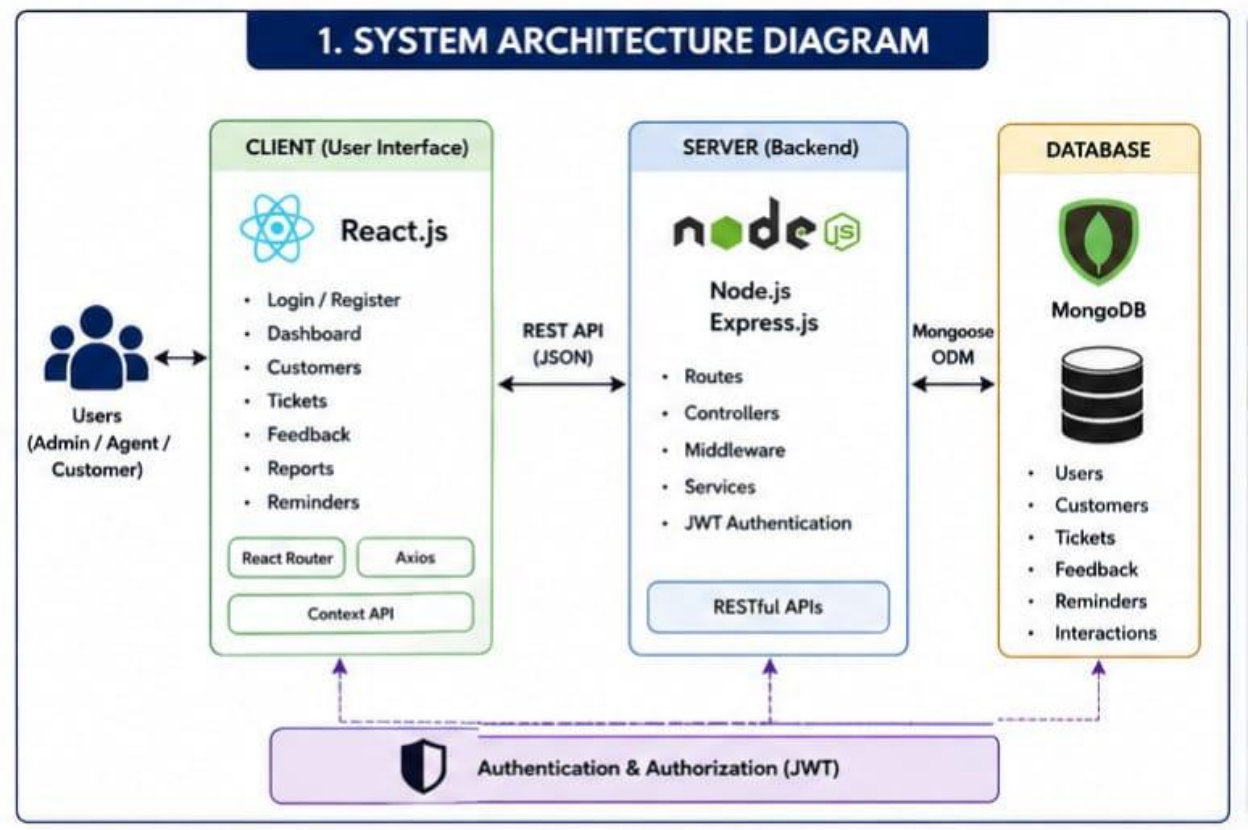
3. Architecture

Frontend (React.js)

Built using React 18 and Vite for fast development and execution. The application provides a responsive user interface with context API managing theme and user auth states.

Components include:

- **Home Page / Dashboard View**
- **Login & Registration Layouts**
- **Customer Registry & Profiles Drawer**
- **Support Tickets Stepper Workspace**
- **Agent Workload Queue Panels**
- **Customer Review CSAT Boards**
- **Advanced Analytics & Diagnostics Control Panel**



Backend (Node.js & Express.js)

The backend is developed utilizing Node.js and Express.js to offer clean, secure REST APIs. Endpoints process operations routing, request handlers, and database connections.

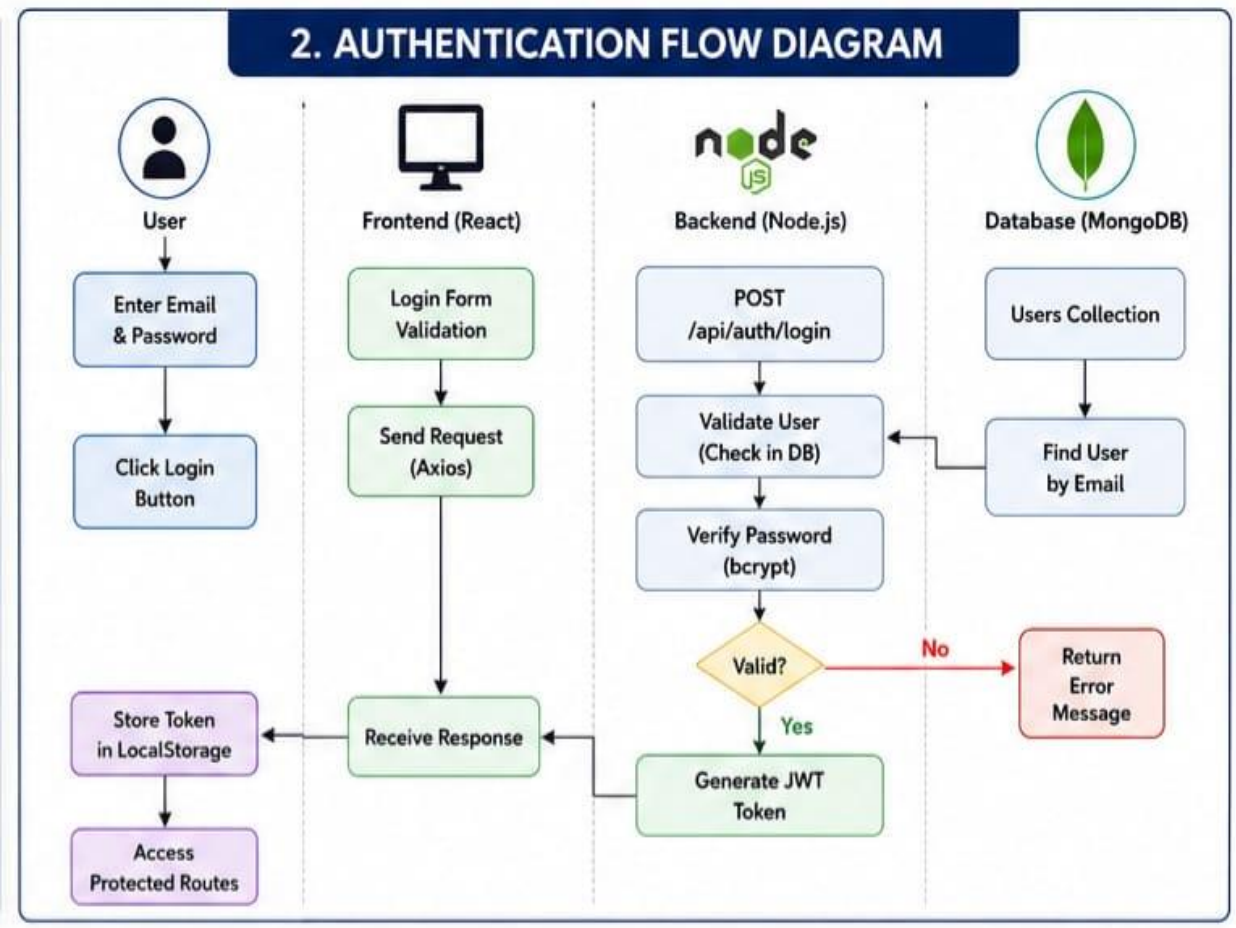
Backend provides REST APIs for:

- **User Authentications & Privileges mapping**
- **Customer profile registration and drawer logs**

- **Support tickets tracking and SLA due timers**
- **Agent assignments and workloads queue tracking**
- **CSAT feedbacks collecting and averaging**

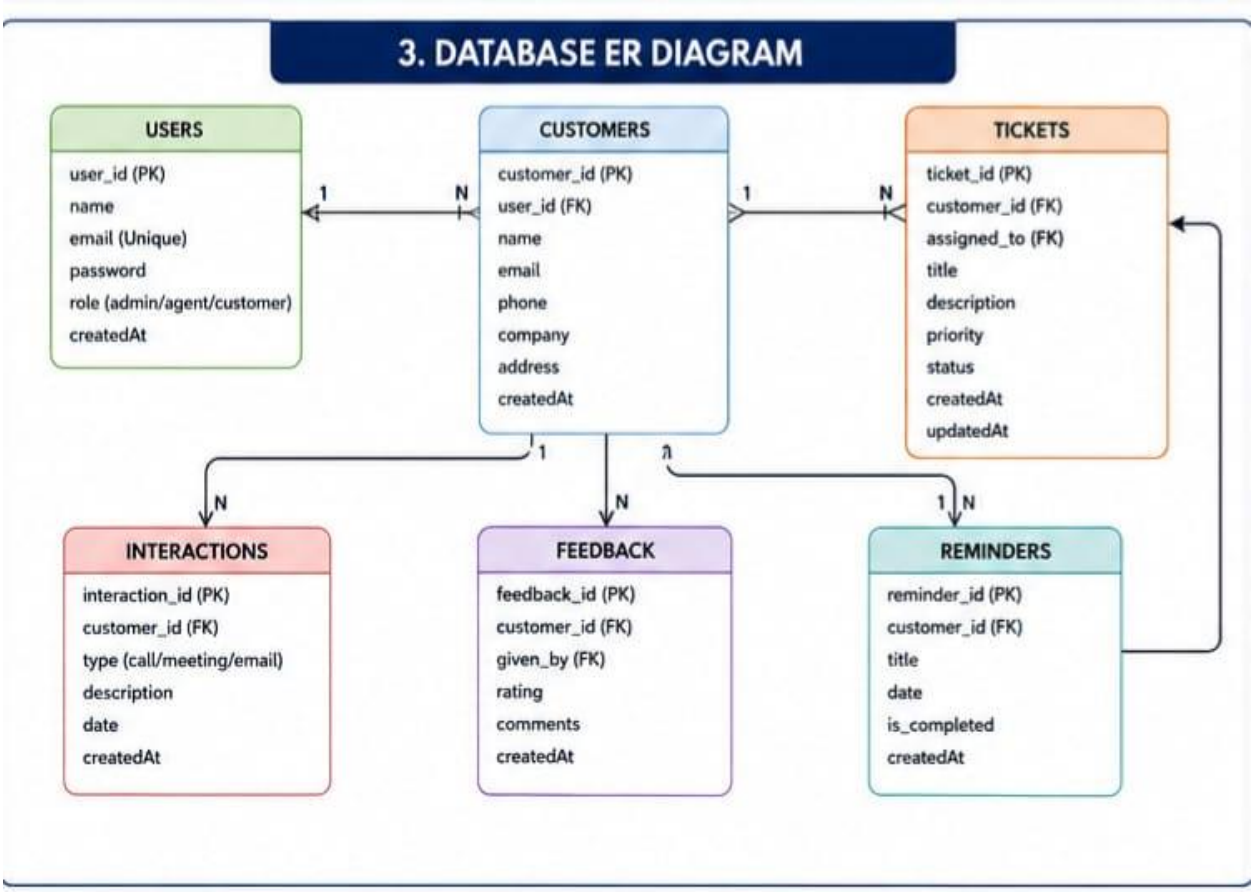
Security includes:

- **JWT (JSON Web Tokens) Session Authentication**
- **bcrypt Password Hashing Encryption**



Database (MongoDB)

MongoDB stores the application collection schemas, including users registry, customer details, note logs, ticketing items, agent queues, and reviews feedbacks.



4. Setup Instructions

Prerequisites

- **Node.js (v18+)**
- **MongoDB Database (Local or Atlas cloud cluster)**
- **VS Code Editor**
- **Git Version Control System**
- **npm Package Manager**

Installation

Clone Project:

```
git clone https://github.com/nikhithasree750-ux/CUSTOMER_REGISTRY.git
```

Install Frontend:

```
cd client
npm install
```

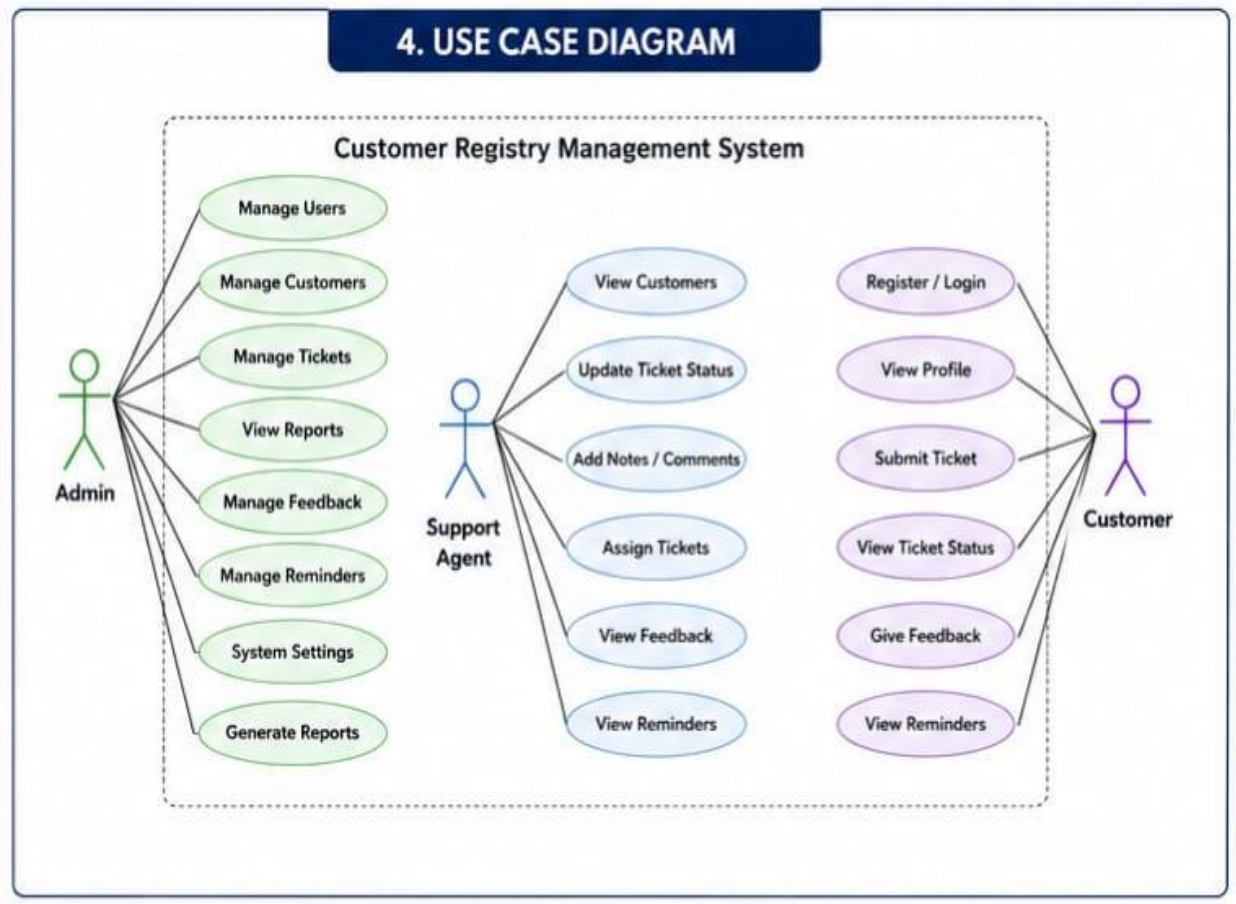
Install Backend:

```
cd ../server
npm install
```

Environment Variables

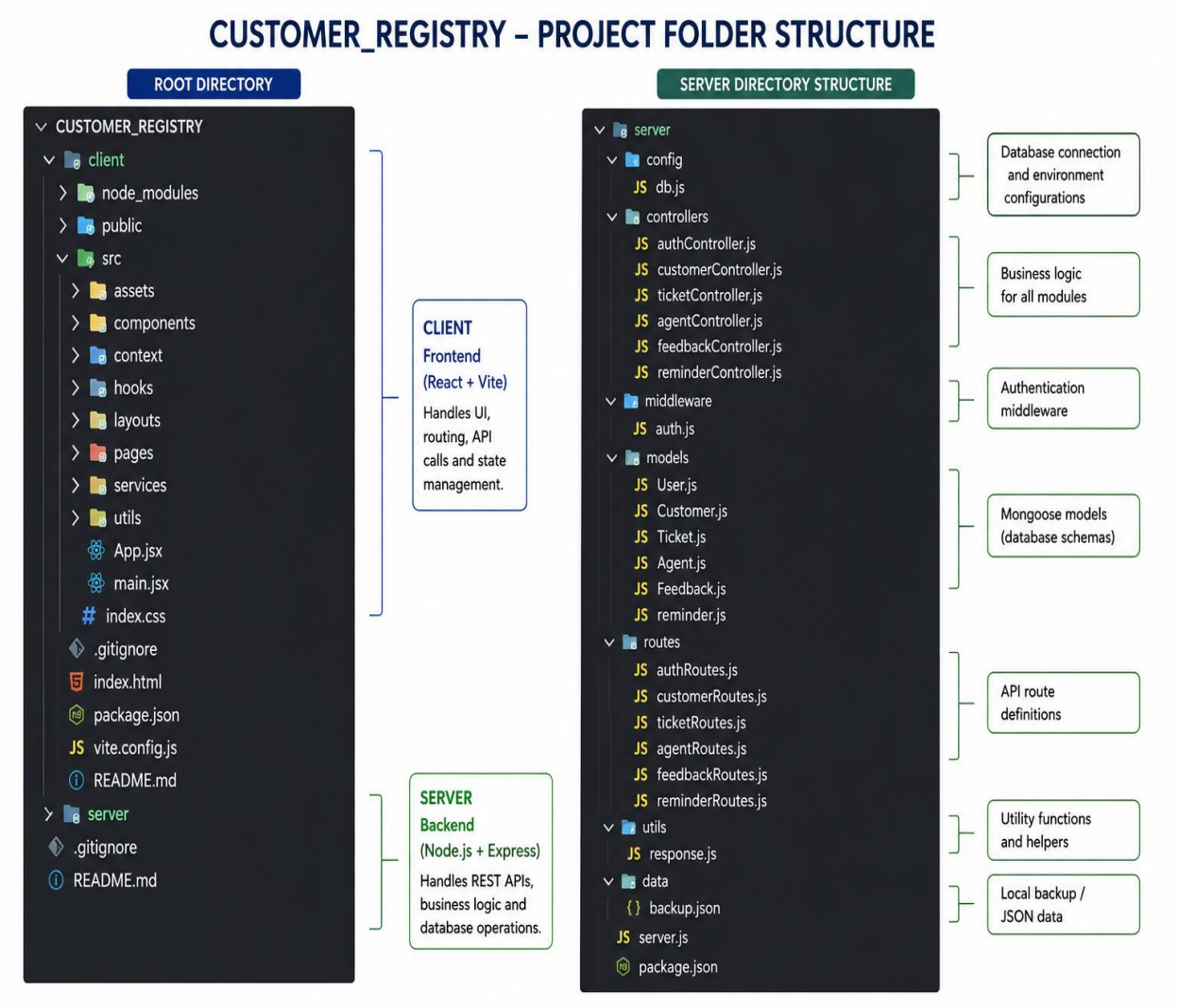
Create a .env file inside the server/ directory:

```
PORT=5001
MONGODB_URI=mongodb://127.0.0.1:27017/customer_registry
JWT_SECRET=SalesFlowSecretJWTCode
```



5. Folder Structure

The project separates client and server operations into independent directory modules. The frontend has UI modules and layouts, while the backend organizes schemas, controller logs, and configurations.



6. Running the Application

To run the Customer Registry Management System locally , start both the backend and frontend servers in separate terminal windows.

Backend Server

- **Open a terminal.**
- **Navigate to the server directory: cd server**
- **Install dependencies: npm install (first time only)**
- **Start the backend server: npm start**
- **The backend server will run at: http://localhost:5001**

Frontend Server

Frontend Server

- **Open a new terminal.**
- **Navigate to the client directory: cd client**
- **Install dependencies: npm install (first time only)**
- **Start the frontend application: npm run dev**
- **The frontend application will run at: http://localhost:5174**

7. API Documentation

The Customer Registry Management System uses RESTful APIs to coordinate data exchange between the React client and Express backend securely.

Method	Endpoint	Parameters / Body	Success Response
POST	/api/users/register	{ name, email, phone, password, role }	{ _id, name, email, phone, role }
POST	/api/users/login	{ email, password } (accepts phone number)	{ _id, name, email, role }
GET	/api/customers	Params: search, status, sort, page, limit	{ customers: [...], pagination: {} }
POST	/api/customers	{ name, email, phone, company, status, ltv }	{ _id, name, company, status, ltv }
PUT	/api/customers/:id	{ company, status, ltv, notes }	{ _id, name, status, ltv }
GET	/api/tickets	Params: status, priority, agentId, search	{ tickets: [...] }
POST	/api/tickets	{ customerId, description, priority }	{ _id, ticketId, priority, dueDate }
PUT	/api/tickets/:id	{ status, assignedAgentId, newComment }	{ _id, ticketId, status, comments }
GET	/api/agents	None	[{ _id, name, specialty, status }]
POST	/api/feedbacks	{ customerId, rating, comment }	{ _id, rating, comment }

8. Authentication & Role-Based Security

Protected by JWT (JSON Web Tokens). Identification verifies credentials using email addresses or mobile phone digits. User roles restrict navigation menus and view layouts:

- **Manager Role:** Elevates user rights to adjust agent details, monitor workloads, and delete client history logs.
- **Agent Role:** Limits users to edit customer directories, logs notes, and update support ticket steppers.

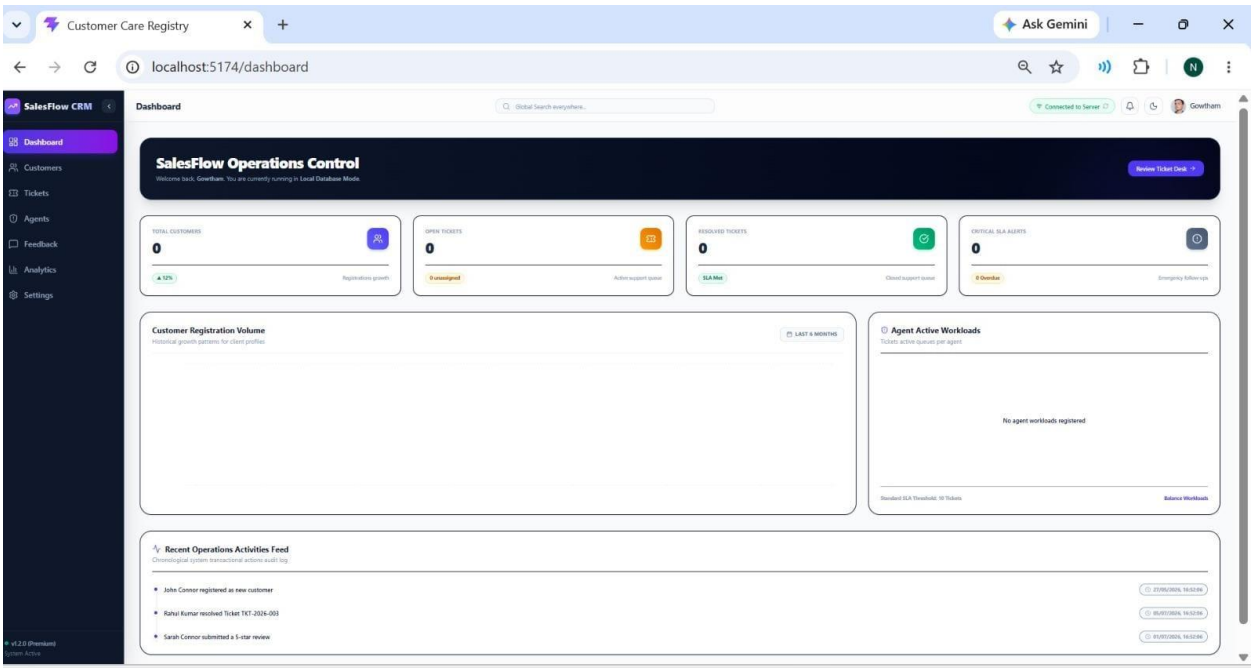
Workflow:-

1. User registers
   ↓
2. Password encrypted (bcrypt)
   ↓
3. Saved into MongoDB
   ↓
4. User Login
   ↓
5. JWT Token generated
   ↓
6. Token sent to frontend (AuthContext)
   ↓
7. Protected API endpoints access

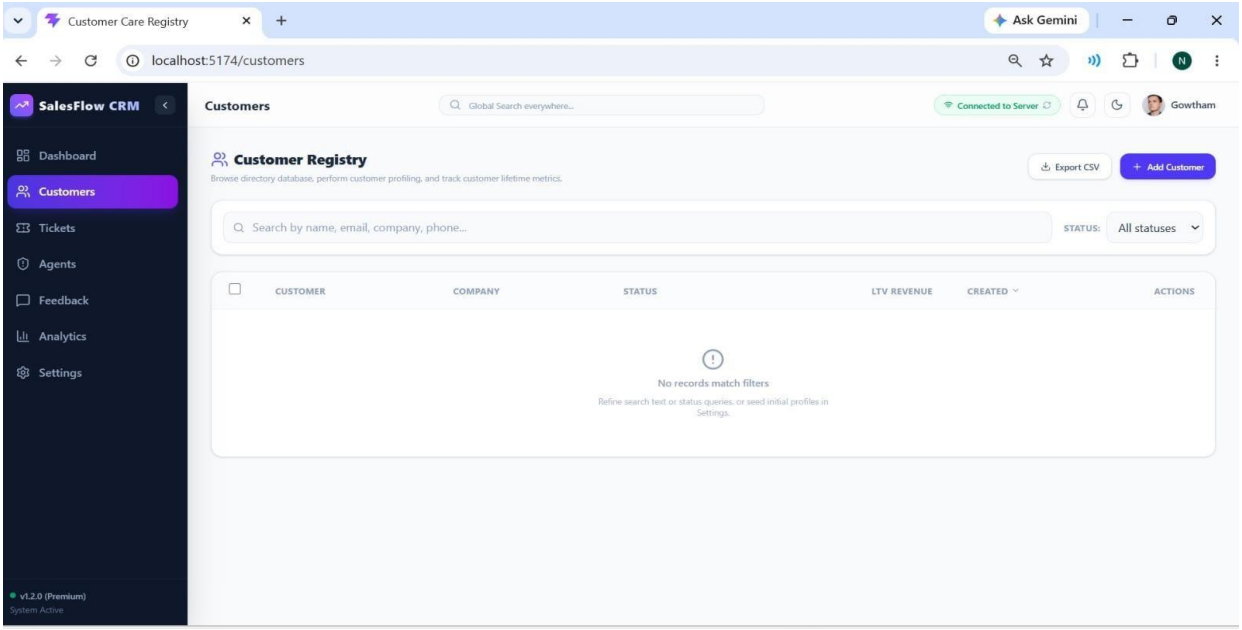
9. User Interface

The Customer Registry Management System user interface is designed using React.js with a responsive and user-friendly layout is designed using React.js with a clean, responsive layout. It provides separate screens for operations and registry control:

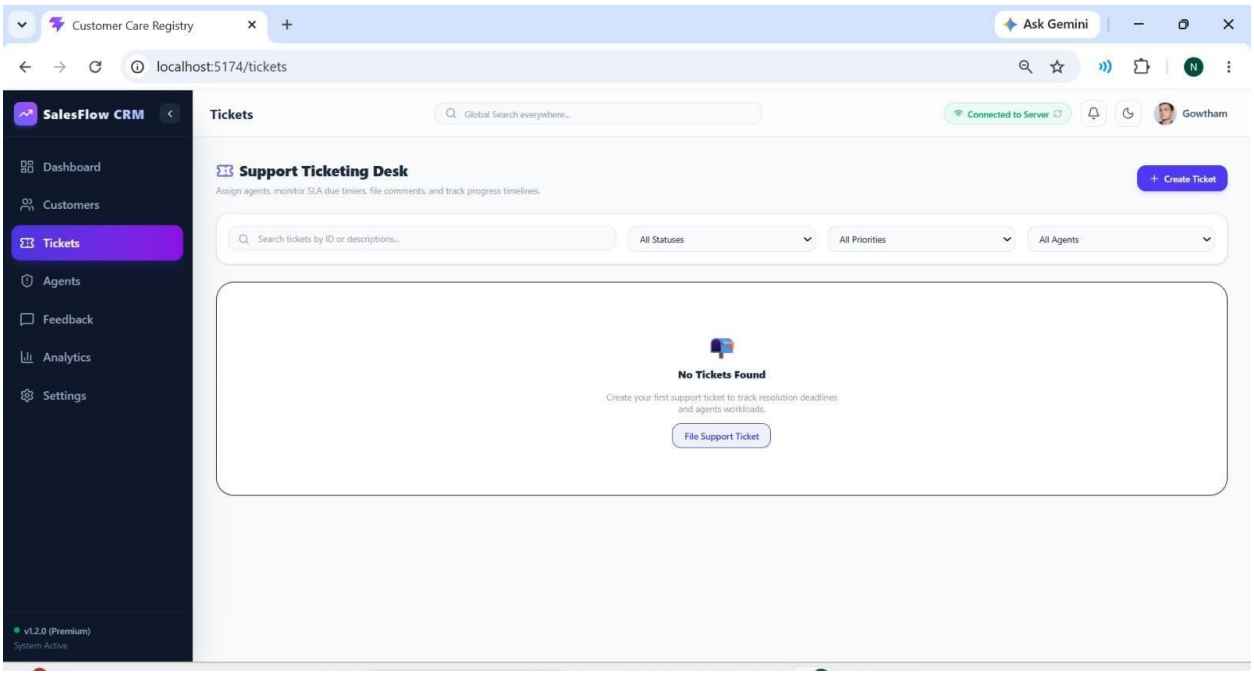
DASHBOARD:-



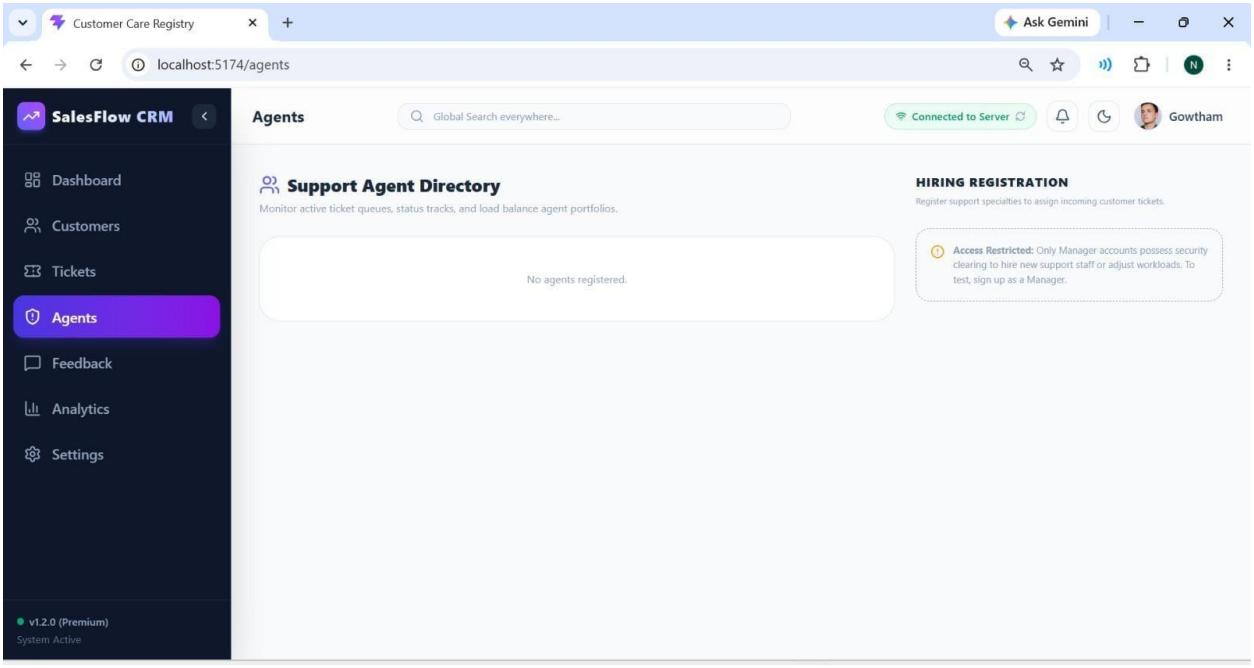
CUSTOMER REGISTRY & DRAWER:-



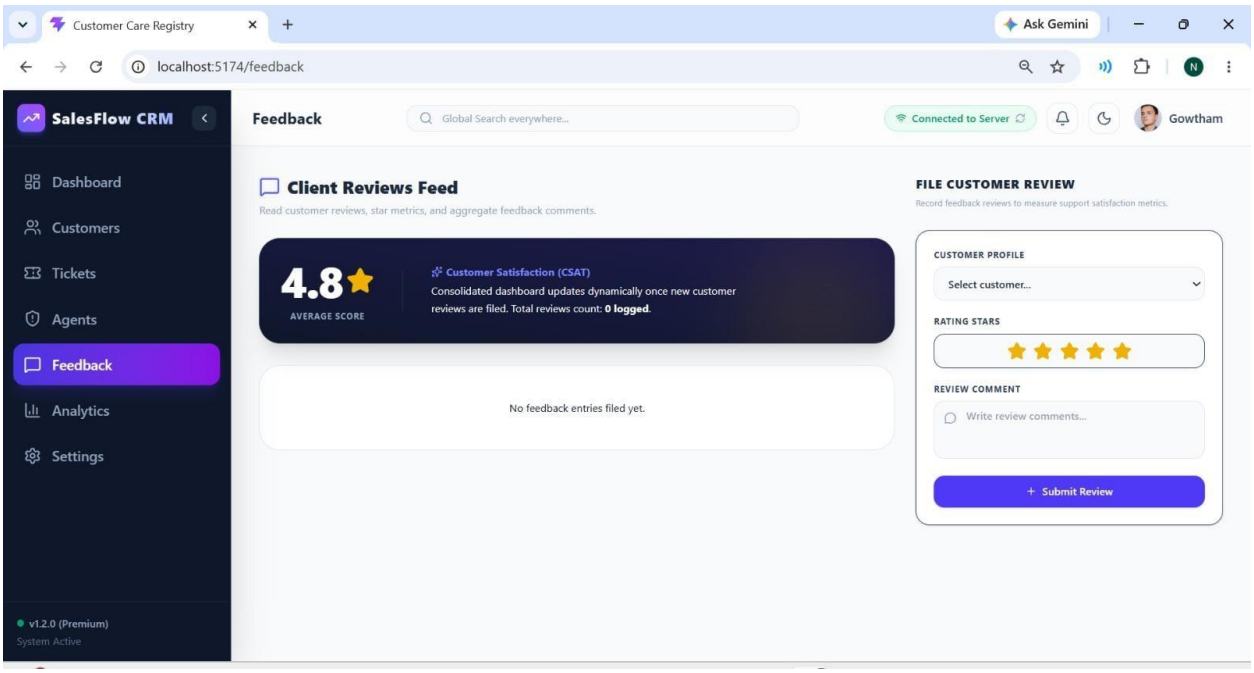
SUPPORT TICKETS DESK:-



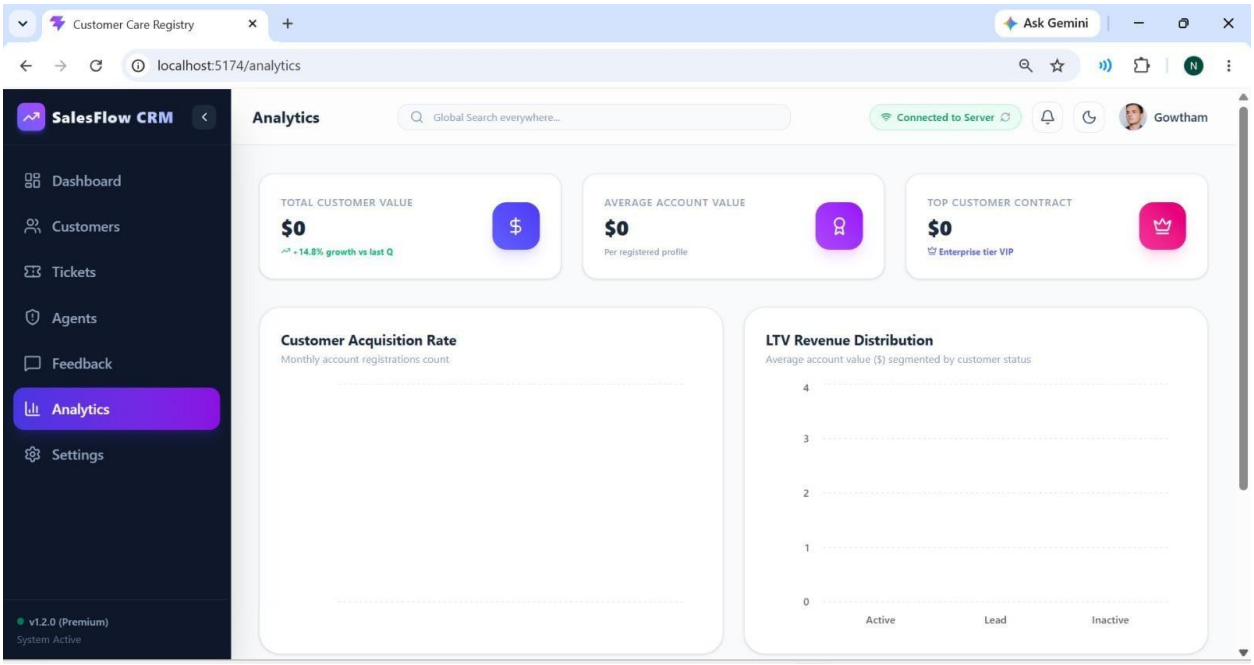
SUPPORT AGENT DIRECTORY:-



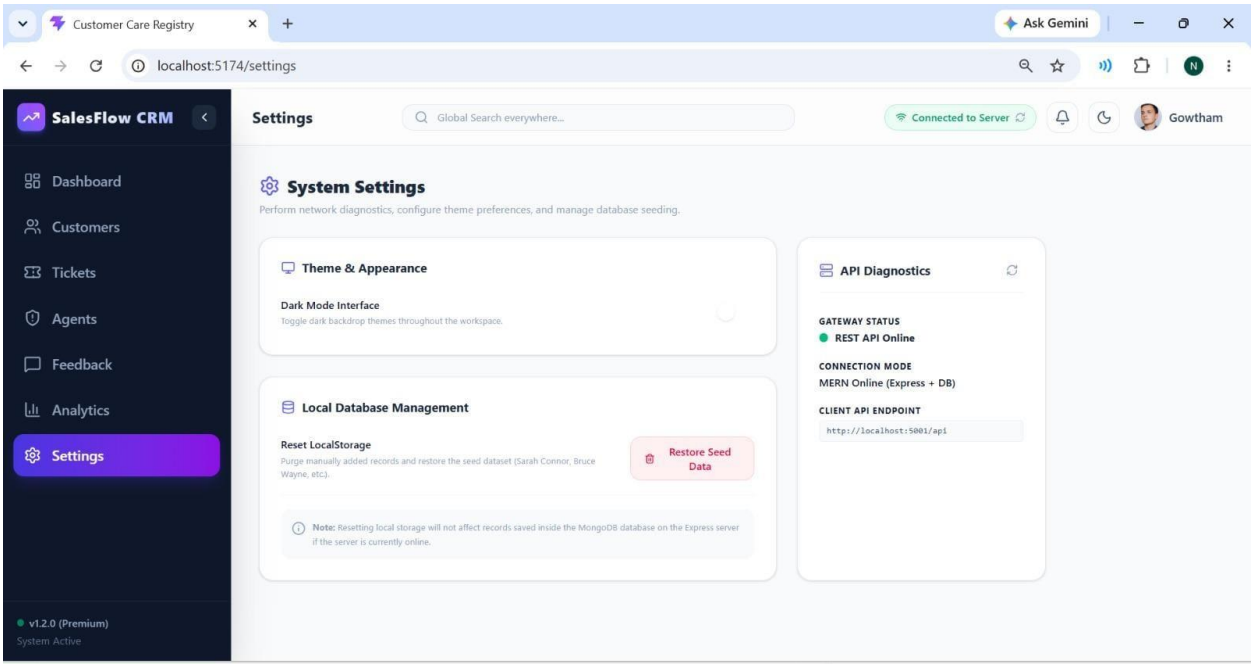
CSAT CUSTOMER REVIEWS BOARD:-



ADVANCED ANALYTICS & LTV REPORTS:-



DIAGNOSTICS & DB SETTINGS PANEL:-



10. Testing & Validation

Testing includes:

- **User Registration Testing**
- **Login Authentication Testing**
- **Support Ticket Operations Testing**
- **Feedback Submissions Testing**
- **Agent Workload Allocation Testing**
- **Database offline fallback switcher Testing**

Testing Type:

- **Functional Testing**
- **Manual Testing**
- **API Testing**
- **Manual Testing**
- **API Testing**

Expected Result:

All modules work successfully.

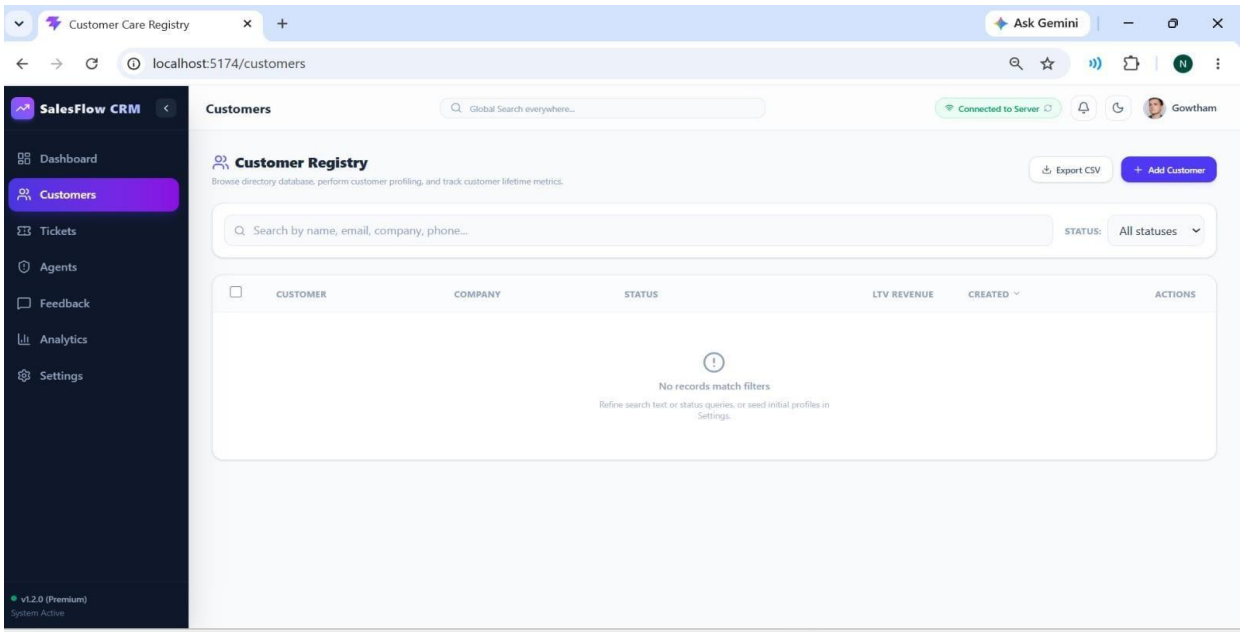
11. Screenshots or Demo

Demo Link:-

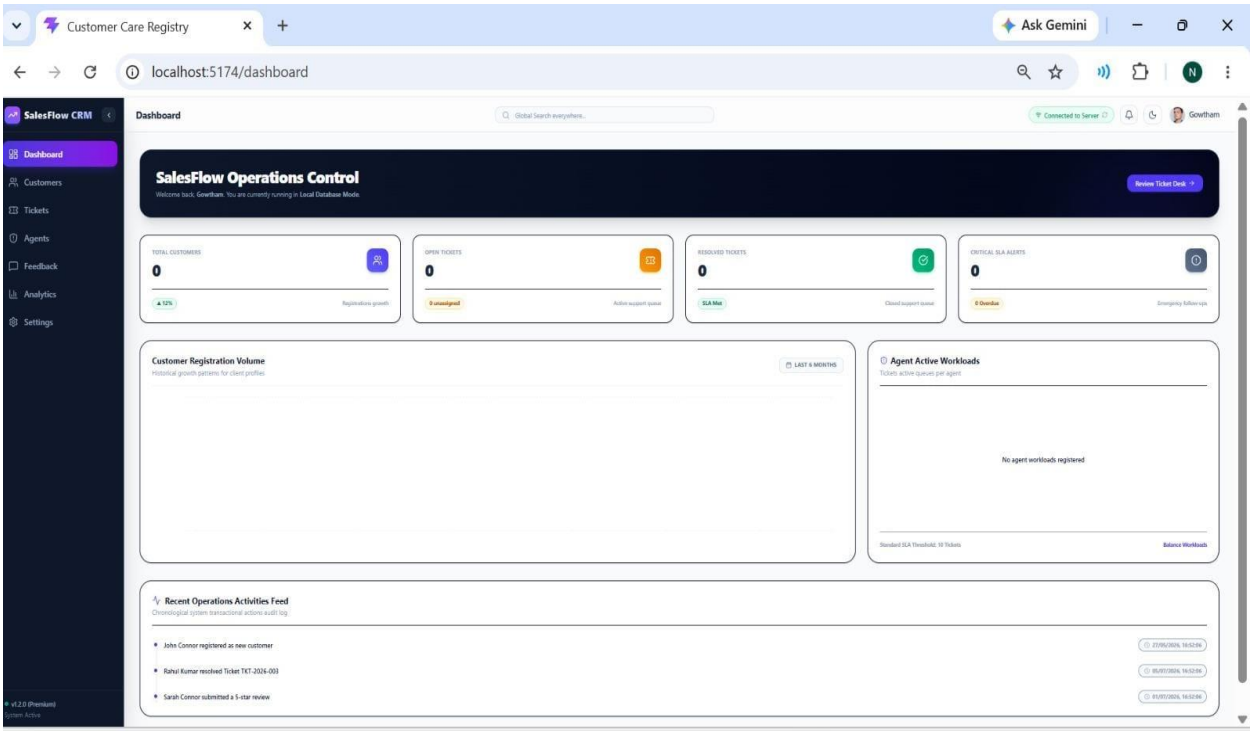
https://github.com/nikhithasree750-ux/CUSTOMER_REGISTRY

Screenshots:-

DASHBOARD:-

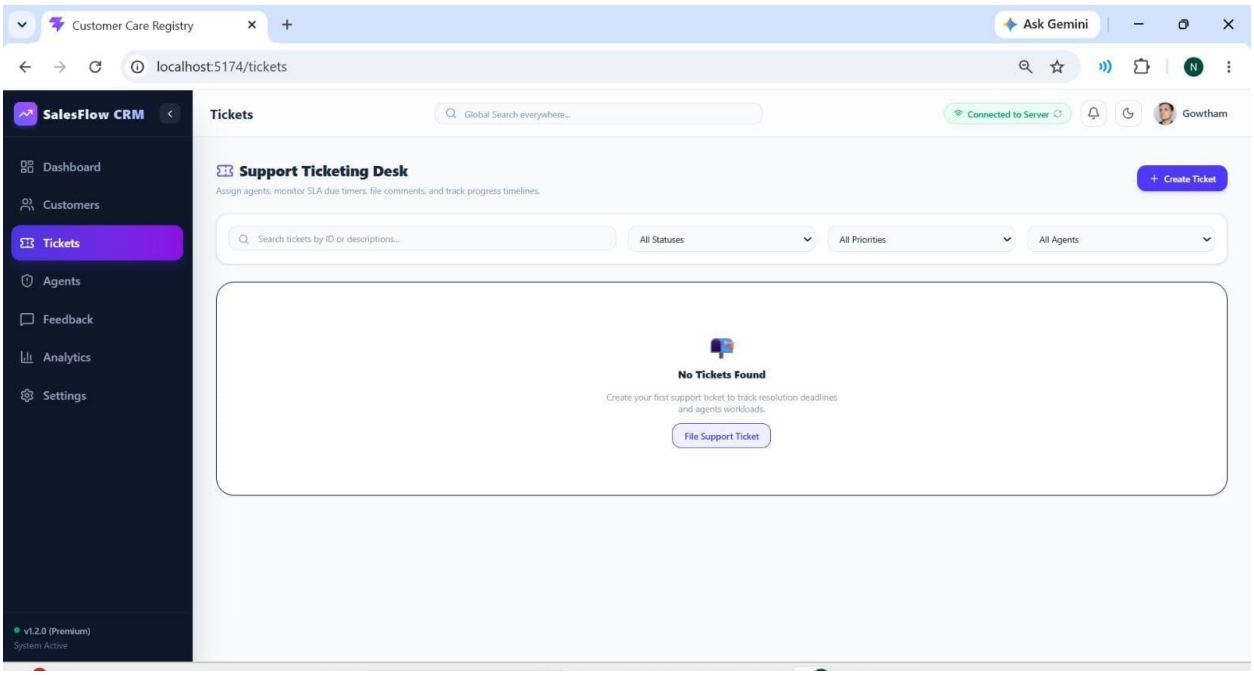


CUSTOMER REGISTRY & DRAWER:

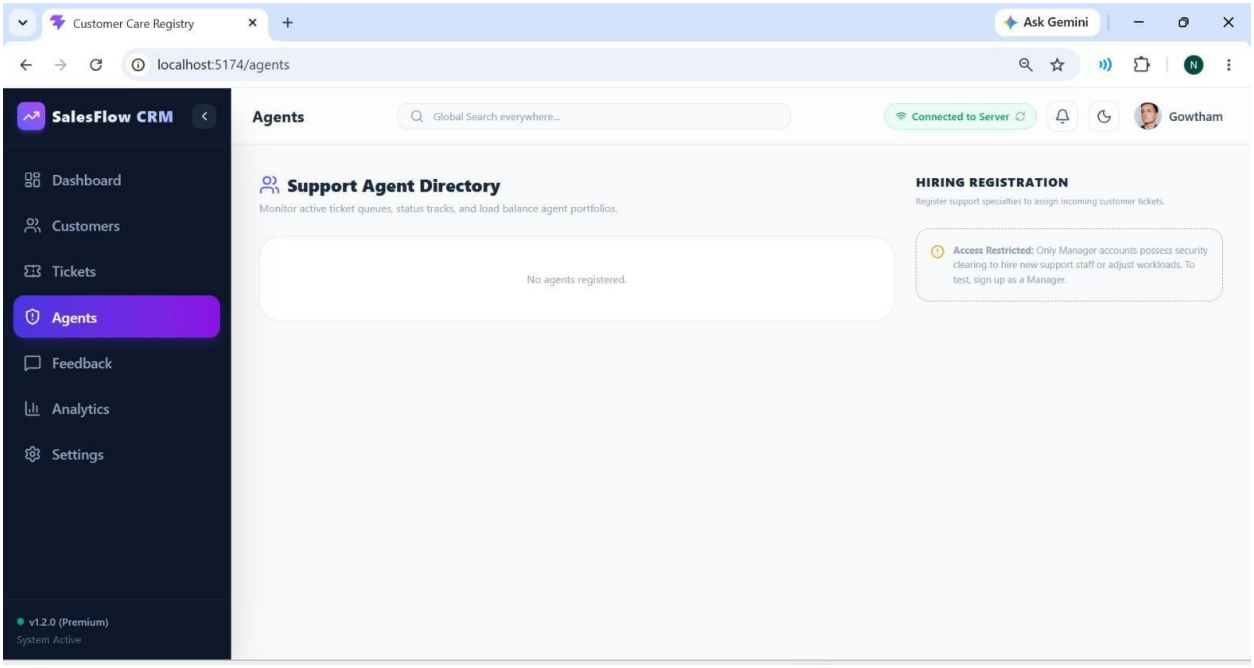


-

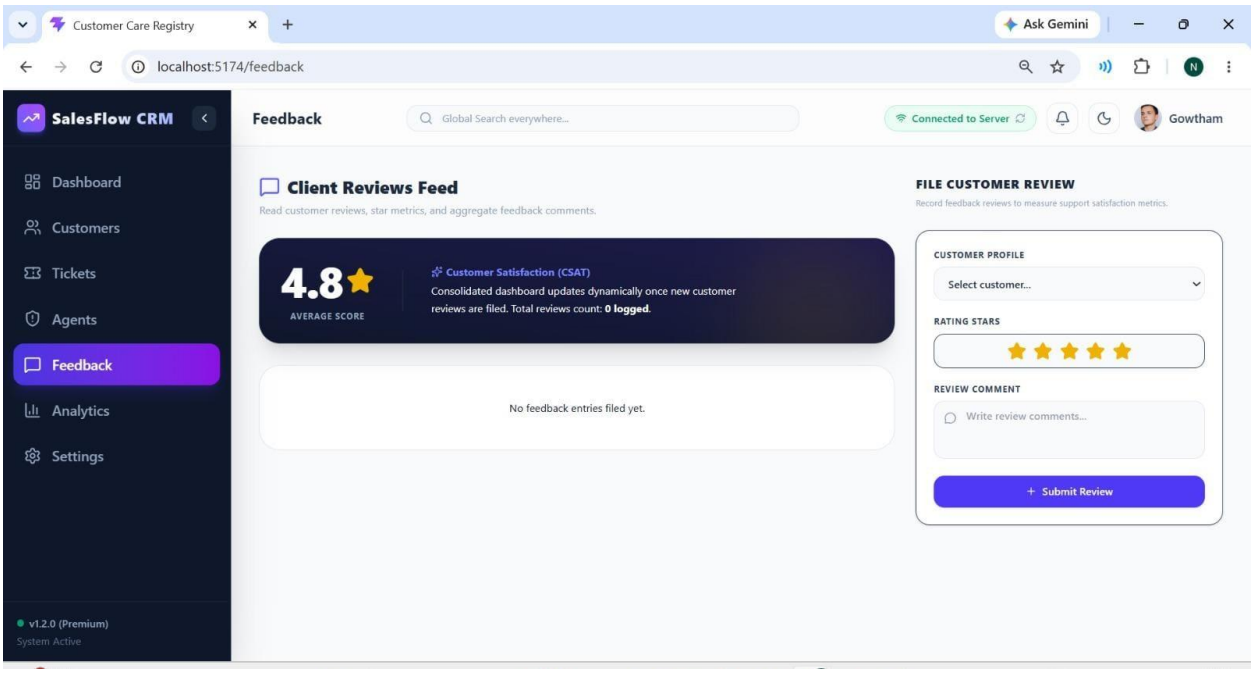
SUPPORT TICKETS DESK:-



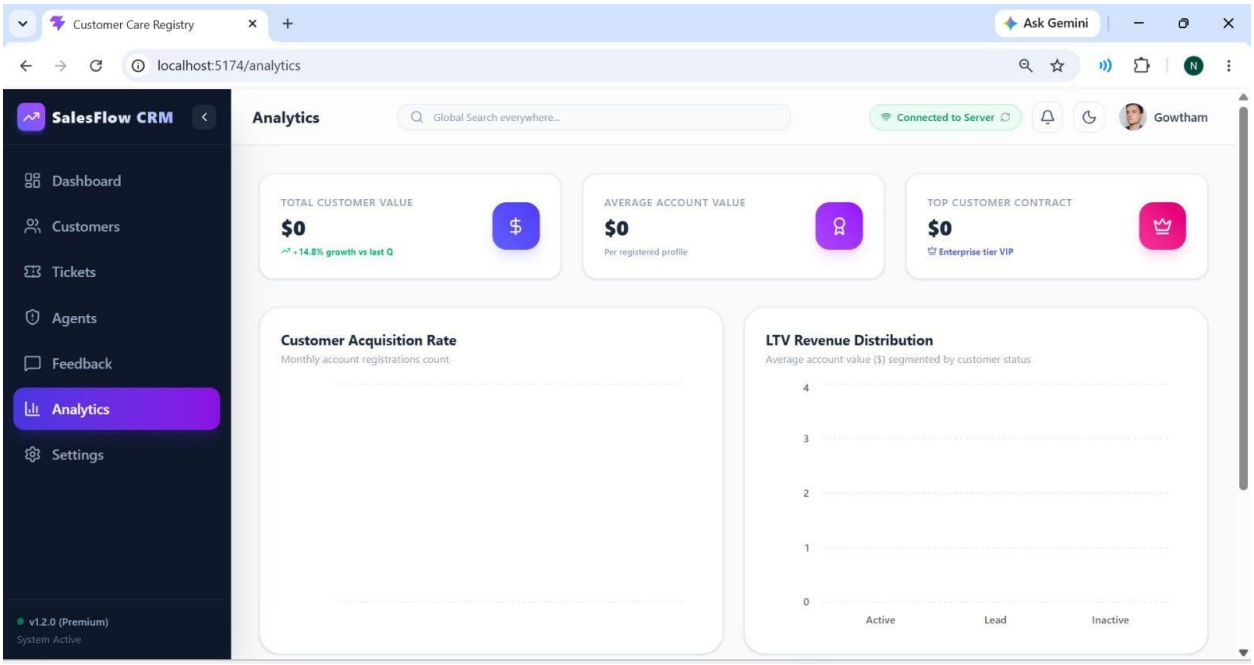
SUPPORT AGENT DIRECTORY:-



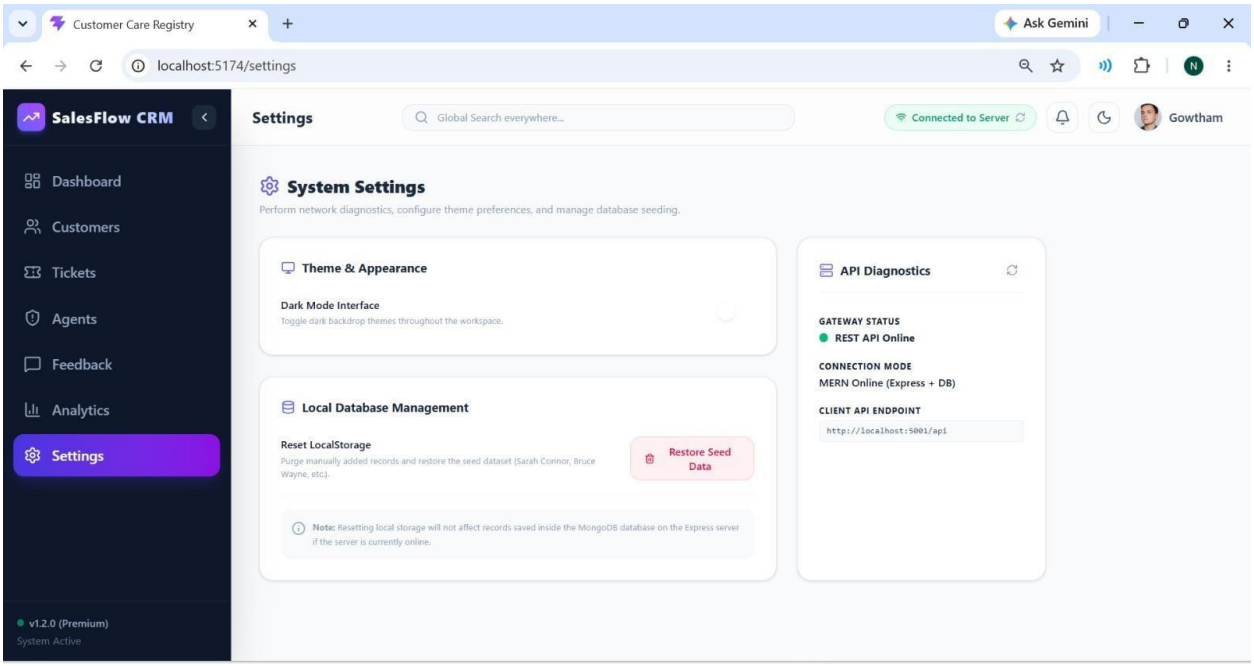
CSAT CUSTOMER REVIEWS BOARD:-



ADVANCED ANALYTICS & LTV REPORTS:-



DIAGNOSTICS & DB SETTINGS PANEL:-



12. Known Issues

- **Windows Port 5000 is occupied by local SSDP Network Discovery. (Resolved by changing Express backend server port to 5001).**
- **Empty charts on client initialization when browser local storage has old structures. (Resolved by clicking 'Restore Seed Data' button in Settings to refresh cache).**
- **Local network request failovers to local storage database cache when frontend is started offline.**

13. Future Enhancements

- **AI-powered chatbot for customer support:** Integrate natural language processing assistants to resolve common customer queries automatically.
- **Email and SMS notification integration:** Notify clients and agents immediately when tickets are created, assigned, or resolved.
- **Mobile application for Android and iOS:** Develop native mobile apps to manage the CRM and support desk on the go.
- **Cloud deployment using AWS or Azure:** Host the MERN stack application on reliable cloud infrastructure for higher availability and scalability.
- **Advanced analytics and business intelligence dashboard:** Implement deep insights, predictive churn analysis, and revenue forecasting charts.
- **Multi-language support:** Localize the user interface to support multiple languages for global operations.

- **Role-based permission management:** Implement granular access controls for managers, agents, and client accounts.
- **Real-time notifications using WebSockets:** Push updates to support boards and activity feeds instantly without manual refreshes.
- **File attachment support for tickets:** Allow users to upload screenshots and logs directly to support tickets.
- **Automated backup and recovery system:** Ensure database integrity with automatic scheduled backups to cloud storage.